🧪 BÀI TẬP THỰC HÀNH

DỰ ĐOÁN KHẢ NĂNG MẮC BỆNH TIỂU ĐƯỜNG

(Binary Classification với Neural Network)

---

1. Giới thiệu bài toán

🎯 Mục tiêu

Xây dựng **một mạng neuron nhân tạo (MLP)** để dự đoán:

Một người **có nguy cơ mắc bệnh tiểu đường hay không**

dựa trên các chỉ số y tế.

📊 Dataset

Pima Indians Diabetes Dataset

- Nguồn: UCI Machine Learning Repository
- Link tải trực tiếp (CSV):
  👉 https://raw.githubusercontent.com/jbrownlee/Datasets/master/pima-indians-diabetes.data.csv

🔍 Mô tả dữ liệu

Thuộc tính	Ý nghĩa
Pregnancies	Số lần mang thai
Glucose	Nồng độ glucose
BloodPressure	Huyết áp
SkinThickness	Độ dày da
Insulin	Insulin
BMI	Chỉ số BMI
DiabetesPedigree	Tiền sử gia đình
Age	Tuổi
Outcome	0: Không mắc, 1: Có mắc

---

2. Yêu cầu bài tập

Sinh viên cần:

1. Đọc và hiểu dataset
2. Tiền xử lý dữ liệu
3. Thiết kế mạng Neural Network
4. Huấn luyện mô hình
5. Đánh giá kết quả
6. Nhận xét và cải tiến

---

3. Hướng dẫn thực hiện (Step-by-step)

🔷 BƯỚC 1: TẢI VÀ KHÁM PHÁ DỮ LIỆU

```
import pandas as pd

url = "https://raw.githubusercontent.com/jbrownlee/Datasets/master/pima-indians-diabetes.data.csv"

columns = [
    'Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness',
    'Insulin', 'BMI', 'DiabetesPedigree', 'Age', 'Outcome'
]

df = pd.read_csv(url, names=columns)
df.head()
```

👉 Câu hỏi cho sinh viên:

- Dataset có bao nhiêu mẫu?
- Có bao nhiêu đặc trưng đầu vào?

🔷 BƯỚC 2: TÁCH INPUT – OUTPUT

```
X = df.drop('Outcome', axis=1)
y = df['Outcome']
```

👉 Giải thích:

- `X`: đặc trưng đầu vào

- `y`: nhãn (0 hoặc 1)

🔷 BƯỚC 3: CHUẨN HÓA DỮ LIỆU

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

👉 Vì sao cần chuẩn hóa?

- Các đặc trưng có đơn vị khác nhau
- Giúp mạng học nhanh và ổn định hơn

🔷 BƯỚC 4: CHIA TẬP TRAIN / TEST

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42
)
```

🔷 BƯỚC 5: THIẾT KẾ MẠNG NEURAL NETWORK

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

model = Sequential()
model.add(Dense(16, activation='relu', input_shape=(8,)))
model.add(Dense(8, activation='relu'))
model.add(Dense(1, activation='sigmoid'))
```

👉 Giải thích:

- Input layer: 8 neuron (8 đặc trưng)
- Hidden layers: ReLU
- Output: 1 neuron + Sigmoid (phân loại nhị phân)

🔷 BƯỚC 6: COMPILE MÔ HÌNH

```
model.compile(
```

```
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)
```

🔷 BƯỚC 7: HUẤN LUYỆN MÔ HÌNH

```
history = model.fit(
    X_train, y_train,
    epochs=50,
    batch_size=16,
    validation_split=0.1,
    verbose=1
)
```

👉 Giải thích:

- Epoch: số vòng học
- Validation: theo dõi overfitting

🔷 BƯỚC 8: ĐÁNH GIÁ MÔ HÌNH

```
loss, acc = model.evaluate(X_test, y_test)
print("Test Accuracy:", acc)
```

👉 Thông thường:

- Accuracy ≈ **75–80%**

🔷 BƯỚC 9: DỰ ĐOÁN THỬ MỘT BỆNH NHÂN

```
import numpy as np

sample = np.array([[2,120,70,20,79,25.0,0.5,33]])
sample_scaled = scaler.transform(sample)

prediction = model.predict(sample_scaled)
print("Nguy cơ mắc bệnh:", prediction[0][0])
```

---

4. Câu hỏi thảo luận

1. Vì sao bài toán này dùng **Sigmoid** ở output?

2. Nếu không chuẩn hóa dữ liệu thì điều gì có thể xảy ra?
3. Thay ReLU bằng Tanh có cải thiện không?
4. Accuracy cao có luôn là tốt không?

---

5. Bài tập mở rộng

🔶 Thử:

- Thay đổi số neuron
- Thêm / bớt hidden layer
- So sánh với Logistic Regression
- Vẽ biểu đồ loss / accuracy theo epoch